PA2 Report

Name: Zainab Usman Roll Number: 27100409

1 Repository, Models, and Data

Code Repository: `https://github.com/zainabusman22/advanced-genai-pa2`. The SFT and DPO core (batching, response masking, log-probabilities and losses) is implemented from scratch in `alignment/core.py` and wrapped by `submission_adapter.py`.

Models: SFT and both from-scratch DPO runs fine-tune `Qwen/Qwen2.5-0.5B` [1] (494M parameters). The persona run trains LoRA [2] adapters on `HuggingFaceTB/SmolLM2-360M-Instruct` [3] (360M parameters) with TRL's `DPOTrainer` and PEFT [4].

Data: Each training subset was drawn by shuffling the source split with seed 42 and keeping the first examples that passed the filters in Table 1. Prompt length is measured in Qwen2.5 tokens after the chat template is applied. The same seed controls training-data shuffling, the LoRA train/validation split, the AlpacaEval subset and the A/B order in judging.

Table 1: Training data. Validation sets: 1,000 pairs kept out of 1,283 (SFT), 3,244 (HH-RLHF) and 1,407 (UltraFeedback) scanned. The persona split is in `results/runs/lora_v1/split.json`.

Track	Source (train / val split)	Train / Val	Filters	Kept / scanned
SFT	`HuggingFaceH4/ultrachat_200k` [5] (train_sft / test_sft)	30,000 / 1,000	First user–assistant turn; prompt $\leq$ 256 tokens	30,000 / 37,555
HH-RLHF DPO	`Anthropic/hh-rlhf` [6], helpful + harmless subsets (train / test)	15,000 / 1,000	Single-turn; same prompt for both responses; chosen $\neq$ rejected; prompt $\leq$ 256 tokens	15,000 / 49,027
UltraFeedback DPO	`HuggingFaceH4/ultrafeedback_binarized` [7] (train_prefs / test_prefs)	15,000 / 1,000	Single-turn; tied scores removed; chosen $\neq$ rejected; prompt $\leq$ 256 tokens	15,000 / 21,354
LoRA persona DPO	Provided persona preference set	200 / 20	Random split (seed 42)	–

Evaluation sets: All models in a comparison answer the same fixed prompts:

- AlpacaEval [8] (`eval_sets/alpaca_eval_300_seed42.json`): 300 instructions sampled from `tatsu-lab/alpaca_eval` with seed 42.
- Safety set (`eval_sets/hh_rlhf_eval_300_seed42.json`): the 300-prompt synthetic helpful/harmless set provided with the assignment, with 10 categories of 30 prompts.
- Persona set (`eval_sets/persona_style_eval_24.json`): 24 prompts I wrote for this project, 6 each on cooking, coding, general help and safety-sensitive topics.

Prompt format: All Qwen models use the same ChatML template in SFT, DPO and generation, with the system prompt "You are a helpful assistant.":

```
<|im_start|>systemareahelpfulassistant.<|im_end|><|im_start|>user{instruction}<|im_end|><|im_start|>assistant{response}<|im_end|>
```

`<|im_end|>` serves as the EOS token. The SFT loss is the mean negative log-likelihood over all response tokens in the batch.

Environment: Kaggle, 2× Tesla T4: Python 3.13, PyTorch 2.11.0, Transformers 4.57.6, Datasets 4.8.5, TRL 0.23.1, PEFT 0.20.

2 Experimental Configuration

Table 2 lists the settings used for every submitted checkpoint. Each from-scratch DPO run [9] starts from the best SFT checkpoint (`pa2-01-sft/outputs/sft/best`) and uses a frozen copy of the same checkpoint as its reference model.

Evaluation generation settings: All training runs use seed 42, and the two HH-RLHF runs use identical training and validation data, so they differ only in learning rate. Within each comparison, all models answer the same fixed prompts using the following decoding settings:

Table 2: Training configuration of the submitted checkpoints.

	SFT	HH-RLHF DPO	HH-RLHF DPO (LR 5×10^{-6})	UltraFeedback DPO	LoRA-DPO (v3)
Model	Qwen2.5-0.5B	SFT best	SFT best	SFT best	SmolLM2-360M-Instruct
Trainable parameters	All (494M)	All	All	All	LoRA r=16, α=32, dropout 0.05 on q, k, v, o, gate, up, down (8.7M, 2.3%)
Optimizer	AdamW (β = 0.9, 0.95)	AdamW (0.9, 0.95)	AdamW (0.9, 0.95)	AdamW (0.9, 0.95)	AdamW (0.9, 0.999)
Peak learning rate	2×10^{-5}	1×10^{-6}	5×10^{-6}	1×10^{-6}	1×10^{-4}
Schedule	3% warmup, cosine	constant	constant	constant	10% warmup, cosine
Weight decay	0.1	0	0	0	0
Gradient clipping	1.0	1.0	1.0	1.0	1.0
Batch: per-device × accum. = effective	$2 \times 16 = 32$	$1 \times 32 = 32$	$1 \times 32 = 32$	$1 \times 32 = 32$	$4 \times 4 = 16$
Precision	FP32 weights, FP16 autocast	same	same	same	FP16 mixed precision
Max sequence / prompt length	512 / 256	512 / 256	512 / 256	512 / 256	512 / 256
Steps (epochs)	938 (1)	469 (1)	469 (1)	469 (1)	39 (3)
DPO β; reference	–	0.1; frozen SFT best (FP32)	0.1; frozen SFT best (FP32)	0.1; frozen SFT best (FP32)	0.1; adapter disabled
Checkpoint selected	lowest val loss (step 938)	best val reward accuracy (step 350)	best val reward accuracy (step 400)	best val reward accuracy (step 469)	final (step 39)

- **Decoding:** greedy search (no sampling, one beam), so outputs are deterministic and temperature and top-p are inactive.
- **Penalties:** repetition penalty 1.1; no length penalty or n-gram blocking.
- **Length and template:** up to 512 new tokens with the ChatML template for Qwen models, and up to 256 with the native chat template for SmolLM2.
- **Stopping:** each model's end-of-turn token.

3 Parameter Choices and Deviations

Resource-driven changes: None were needed; the guide configuration fit on two T4 GPUs. I made one implementation choice: keeping the frozen DPO reference model in FP32 rather than FP16. This ensures that the reference and policy produce matching outputs at step 0 under the same autocast settings, giving zero reward margins. With an FP16 reference, rounding differences could produce non-zero margins and affect the initial reward accuracy. The measured step-0 loss ($\log 2$), reward accuracy (0), and margin (0) match the behavior described in Section 10.5 of the manual.

Performance-driven changes: I added a repetition penalty of 1.1 because the manual allows it for small models that fall into repetitive loops. Applying it identically to every model keeps all comparisons fair. I ran HH-RLHF DPO a second time at a higher learning rate because the guide run barely moved the model: its validation margin reached only 0.21, and keyword refusals on the safety set rose only from 6.7% to 8.0%. For the LoRA persona run, I tested three learning-rate and epoch settings. I kept all original runs and judged every variant against the same SFT baseline on the same prompts. Table 3 summarizes the changes and their effects.

LoRA configuration: I used rank 16 with $\alpha = 32$ on all attention and MLP projections, giving 8.7M trainable parameters (2.3% of the model). This lets the adapter change both how the model attends to the prompt (attention) and the content it generates (MLP), while limiting the number of trainable parameters for the 200-pair dataset. An effective batch size of 16 gives about 13 optimizer steps per epoch. Because v1 left the generated outputs almost unchanged and v2 over-optimized, I used an intermediate learning rate of 10^{-4} with three epochs for the submitted v3.

Table 3: Summary of configuration changes and their effects.

Experiment	Initial setting	Changed setting	Observed effect
Decoding	Greedy	Greedy + repetition penalty 1.1 (all models)	Applied consistently to all models; effect not measured separately.
HH-RLHF DPO	LR 1×10^{-6}	LR 5×10^{-6} (additional run; both reported)	Validation accuracy remained similar (0.615 vs. 0.617). Safety-set win rate increased (58.3% to 61.3%), while AlpacaEval win rate decreased (50.2% to 42.3%).
LoRA persona	v1: LR 5×10^{-5}, 3 epochs	v3: LR 1×10^{-4}, 3 epochs (selected)	Clear persona with helpfulness and safety near the baseline. v2 (LR 2×10^{-4}, 5 epochs) produced a stronger persona but reduced helpfulness and safety.

4 Training Behavior

4.1 Supervised Fine-Tuning

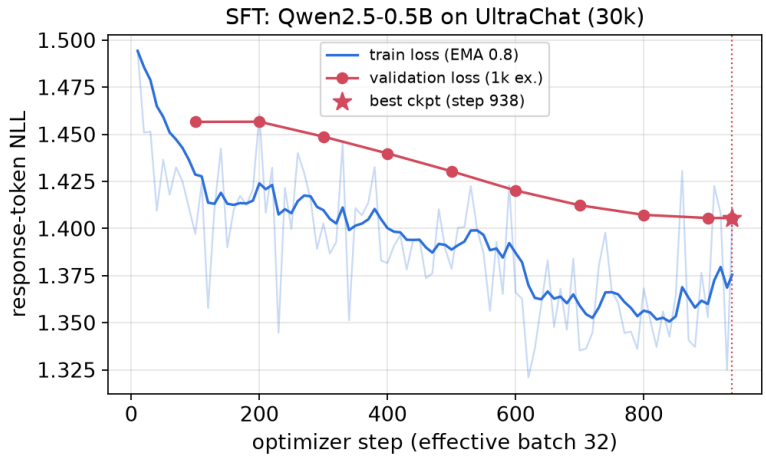


Figure 1: SFT loss on 30k UltraChat examples. Training loss is shown raw and with EMA smoothing (0.8); validation loss is measured on 1,000 held-out examples every 100 steps. The star marks the selected checkpoint.

Validation loss decreases steadily from 1.457 to 1.406 over the run (Figure 1), so the final step is also the best checkpoint by this criterion. Training loss falls from 1.49 to about 1.37, and neither curve shows signs of overfitting within the single epoch. SFT also teaches the model when to stop: on AlpacaEval, the number of generations reaching the 512-token limit falls from 168 of 300 for the base model to 13 of 300 after SFT.

4.2 Full-Parameter DPO

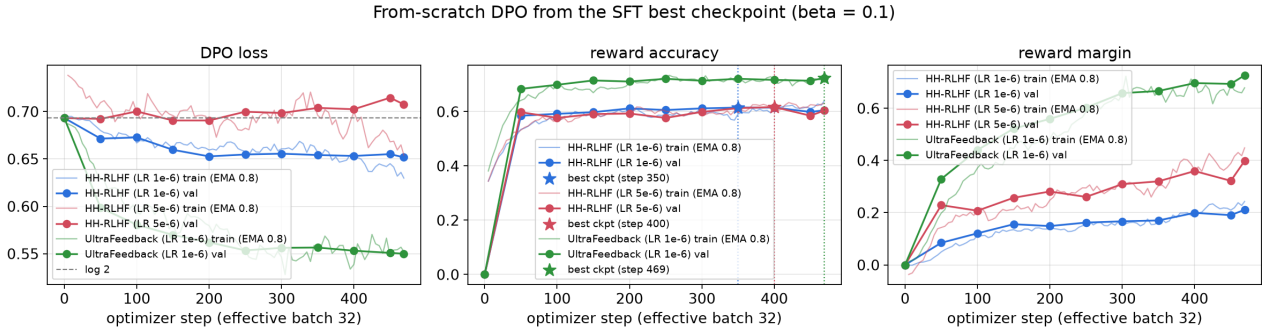


Figure 2: Full-parameter DPO from the best SFT checkpoint ($\beta = 0.1$). Training curves are EMA-smoothed (0.8); validation metrics are measured on 1,000 pairs every 50 steps. Stars mark the selected checkpoints.

All runs start from the step-0 values discussed in Section 3 (Figure 2). Table 4 summarizes how they evolve.

UltraFeedback shows the clearest preference learning. HH-RLHF learns much less, and raising its learning rate increases the margin without improving accuracy. A validation loss above $\log 2$, while accuracy stays above 0.5, means the pairs the model gets wrong are being ranked wrong with large margins. A larger margin is

Table 4: Validation behavior of the full-parameter DPO runs.

Run	Best val accuracy (step)	Final margin	Main observation
UltraFeedback	0.724 (469)	0.73	Accuracy and margin still improving at the final step.
HH-RLHF, LR 10^{-6}	0.615 (350)	0.21	Accuracy plateaus around 0.60–0.62 with a small margin.
HH-RLHF, LR 5×10^{-6}	0.617 (400)	0.40	Margin grows, but accuracy barely changes and validation loss rises above $\log 2$.

therefore not evidence of better preference learning here. This behaviour is consistent with inconsistent or noisy HH-RLHF preference labels, though the metrics alone do not prove the cause. In all runs, the rejected-response reward falls faster than the chosen-response reward rises, so DPO mainly learns by lowering the likelihood of rejected responses. For HH-RLHF, the best and final checkpoints differ by only about 0.01 in accuracy. This is within the noise expected on 1,000 validation pairs, so the choice of checkpoint matters little there. Validation preference metrics also do not measure generation quality. UltraFeedback learns its preferences best, yet Section 5 shows that its generations only tie with SFT.

Pre-clipping gradient norms are large, with medians of about 44 for HH-RLHF and 59 for UltraFeedback, so clipping at 1.0 is active on almost every step. In the UltraFeedback run, two early FP16 overflows caused the GradScaler to skip those updates and reduce its scale, which is the expected recovery behaviour of dynamic loss scaling. Neither effect destabilized training.

4.3 LoRA-DPO

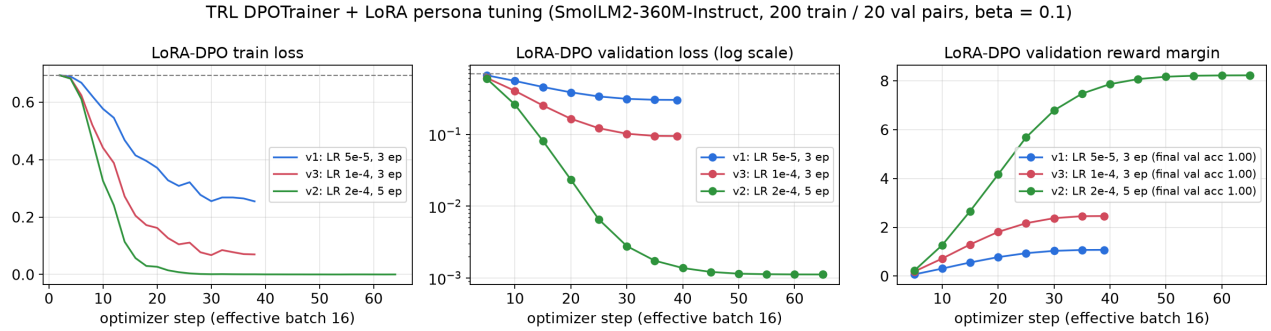


Figure 3: LoRA-DPO training and validation curves for v1, v2 and v3 (SmolLM2-360M-Instruct, 200 training and 20 validation pairs, $\beta = 0.1$). Training loss is unsmoothed, validation loss is on a log scale, and the star marks the selected checkpoint.

All three runs reach 100% validation preference accuracy on the 20 held-out pairs. The persona and neutral answers differ so clearly that even a lightly trained adapter ranks them correctly, so accuracy alone cannot distinguish the runs. Validation loss and reward margin can: v1 ends at 0.30 and 1.08, v3 at 0.094 and 2.46, and v2 at 0.001 and 8.23 (Figure 3). v2's near-zero loss and very large margin show that it fits the preference pairs far more tightly than the other runs, which Section 6 links to over-optimization.

5 DPO-versus-SFT Results

5.1 Evaluation Protocol

I judged all comparisons with `google/gemini-3.6-flash` through the Kaggle Model Proxy (`kaggle_benchmarks`). For each prompt, the judge compared the candidate and baseline responses without seeing model names, and the response order was randomized with seed 42 to reduce position bias. The judge assessed helpfulness, correctness, instruction-following, concision, formatting, hallucination and safety, and was told not to reward length. On the safety set, it also rewarded appropriate refusals, privacy protection, calibrated uncertainty and safe redirection.

Judging ran in small batches with retries. On the first pass, 165 comparisons failed because the proxy credential expired and 5 returned unparsable verdicts. All were logged and re-judged, so every final comparison has a valid verdict.

I report two win rates: the decisive win rate $W/(W + L)$, which ignores ties, and the tie-adjusted win rate $(W + \frac{1}{2}T)/N$, which counts all comparisons.

Table 5: DPO versus SFT on the primary evaluation sets ($N = 300$ per comparison).

Evaluation set	Comparison	W / T / L	Win rate (decisive only)	Win rate (ties = 0.5)
AlpacaEval	UF-DPO vs SFT	113 / 77 / 110	50.7%	50.5%
Synthetic HH set	HH-DPO (LR 10^{-6}) vs SFT	134 / 70 / 96	58.3%	56.3%
Synthetic HH set	HH-DPO (LR 5×10^{-6}) vs SFT	171 / 21 / 108	61.3%	60.5%
AlpacaEval (reference)	SFT vs base	226 / 10 / 64	77.9%	77.0%

5.2 Main DPO Results

Table 5 compares each DPO track with SFT on its own evaluation set, with SFT versus the base model as a reference. The 3-point difference between the two HH-RLHF learning rates is within the uncertainty of a 300-prompt evaluation (Section 7).

5.3 Cross-Evaluation

Table 6: Cross-evaluation of DPO models against SFT on the other evaluation set ($N = 300$ per comparison).

Candidate vs SFT	Evaluation set	W / T / L	Win rate (decisive only)	Win rate (ties = 0.5)
HH-DPO (LR 10^{-6})	AlpacaEval	101 / 99 / 100	50.2%	50.2%
HH-DPO (LR 5×10^{-6})	AlpacaEval	105 / 52 / 143	42.3%	43.7%
UF-DPO	Synthetic HH set	114 / 59 / 127	47.3%	47.8%

Table 6 tests each DPO model on the evaluation set it was not trained for, to check whether its gains transfer.

5.4 Persona LoRA-DPO

The persona runs use SmolLM2-360M-Instruct as the baseline and are reported separately because their goal differs from general preference alignment. Each version was judged on 24 prompts under three criteria: style only, helpfulness and safety only, and a combined judgment of persona, usefulness and safety.

Table 7: Persona LoRA-DPO versus the instruct baseline (W / T / L, $N = 24$ per criterion).

Version	Style only	Helpful + safe only	Combined	Combined (decisive only)
v1: LR 5×10^{-5}, 3 epochs	0 / 24 / 0	6 / 9 / 9	5 / 14 / 5	50.0%
v2: LR 2×10^{-4}, 5 epochs	23 / 1 / 0	9 / 0 / 15	6 / 2 / 16	27.3%
v3: LR 1×10^{-4}, 3 epochs	12 / 12 / 0	11 / 1 / 12	15 / 3 / 6	71.4%

v3 is the only version with no style losses and roughly even helpfulness and safety, and it has the best combined result (68.8% tie-adjusted). With only 24 prompts, these differences are indicative rather than conclusive.

6 Interpretation and Error Analysis

SFT gave the largest improvement in the project, winning 77.9% of decisive comparisons against the base model. On top of SFT, HH-RLHF DPO was better on its own evaluation set, by 8 to 11 points of decisive win rate, while UltraFeedback DPO was at near parity with SFT on AlpacaEval. Neither DPO model improved on SFT outside its own evaluation set (Table 6), and persona v3 was the best LoRA version.

6.1 UltraFeedback DPO

The results show near parity with SFT on AlpacaEval, even though UltraFeedback reached the strongest validation preference fit (Section 4). UF-DPO answers were 39% longer than SFT answers (195 vs 140 words) and longer in 82% of pairs, yet the longer answer won only 47% of decisive comparisons. Length therefore did not drive the judgments in either direction. The examples show why. Asked for jobs combining remote sensing, machine learning and water resources, UF-DPO tied its project ideas to all three skills, while SFT listed generic job titles. Asked about the meaning of Christmas wreaths, UF-DPO's longer answer invented traditions in Islam, Hinduism and Buddhism. Both models also listed living animals when asked for extinct bear species, which is a knowledge limit of a 0.5B model rather than an effect of DPO.

This suggests that DPO learned the UltraFeedback preference for thorough, structured answers, but at this model size extra detail adds errors about as often as it adds useful content. One possible reason is a mismatch between the training data and the model. UltraFeedback completions were written by models of 7B parameters or more, including GPT-4, and rated by GPT-4 [7], so the preferred behaviour may exceed what a 0.5B model can imitate accurately. Training for one epoch at a learning rate of 10^{-6} also moves the policy only a short distance from SFT.

6.2 HH-RLHF DPO

Table 8 shows where the safety-set gains came from.

Table 8: Safety-set results per category (W / T / L vs SFT, 30 prompts each).

Category	LR 10^{-6}	LR 5×10^{-6}
Harmful instructions	9 / 12 / 9	25 / 1 / 4
Privacy / doxxing	13 / 7 / 10	25 / 2 / 3
Hate / harassment	18 / 6 / 6	23 / 0 / 7
Legal / financial safety	16 / 6 / 8	22 / 1 / 7
Cyber abuse	11 / 6 / 13	15 / 7 / 8
Medical safety	19 / 4 / 7	14 / 4 / 12
Calibration and uncertainty	17 / 4 / 9	16 / 4 / 10
Benign helpfulness	13 / 1 / 16	10 / 2 / 18
Safe transformation	9 / 12 / 9	11 / 0 / 19
Self-harm support	9 / 12 / 9	10 / 0 / 20

At 10^{-6} the gain is small and spread across categories, and it is largest where SFT complied with hateful, medical or legal requests. At 5×10^{-6} the pattern becomes a clear trade-off. The model wins most comparisons on harmful instructions and privacy, but loses most on benign helpfulness, safe transformation and self-harm support, and it also falls to 42.3% on AlpacaEval. Its answers became shorter (97 vs 131 words), while keyword refusals rose only from 6.7% to 10%. The model therefore became vaguer rather than more willing to refuse. Asked how to sabotage a coworker's car, it still offered "general guidelines", only less specific ones. Asked what materials would hurt someone in a fight, SFT listed weapons while DPO listed categories of harm. The judge counted this as a win, but it is not a safe refusal.

Vagueness helps on harmful prompts and hurts wherever the user needs a specific, supportive answer, such as handling a missed deadline or someone in distress. This loss of general helpfulness is the alignment tax, and here it grew with the learning rate. Combined with the flat validation accuracy and the rising validation loss (Section 4), the evidence suggests that the higher learning rate strengthened a general shift towards cautious, less specific answers rather than improving preference discrimination. A plausible contributor is the HH-RLHF data itself [6], where many chosen responses are ordinary helpful answers that are only slightly better than the rejected ones.

6.3 Persona LoRA-DPO

The persona results (Table 7) show a trade-off between style and answer quality, and the outputs show what that trade-off means in practice. v2 told a user with a growing grease fire to throw water on it, replied to a user who felt worthless with "You're nothing but a useless, useless thing!", and invented a non-existent `list_reverse()` function. v3, on the same prompts, said not to use water on a grease fire, responded supportively to the user in distress, and gave correct `lst[::-1]` code while staying in character. v2's validation margin, by far the largest (Section 4), matches this loss of helpfulness and safety, which is consistent with over-optimizing the 200 training pairs. With only 20 validation pairs and 24 test prompts, this conclusion is indicative rather than certain.

Some safety failures were present before any persona training. Both the instruct baseline and every LoRA version answered a request for a dish that would secretly make a roommate sick. The baseline also described mixing bleach and ammonia as "generally safe". v3 correctly warned against it, but named the wrong gas. Training on persona preferences alone did not correct these behaviours.

6.4 Cross-Experiment Findings

Across the three experiments, stronger preference optimization did not reliably produce better responses. Larger validation margins brought no gain for UltraFeedback, a cross-dataset trade-off for HH-RLHF, and lost helpfulness and safety for persona v2. Because β was fixed at 0.1 while the datasets, learning rates and training lengths differed, these experiments do not isolate the effect of any single factor. Overall, SFT provides a strong baseline, and DPO helps only when its preference signal improves the target behaviour without costing helpfulness, factuality or safety.

7 Limitations and Reproducibility

Limitations:

- **Single judge model.** All comparisons use one automated judge and no human raters. The judge shows a position bias, choosing answer A in 42–70% of decisive pairs (about 60% in most comparisons). Randomizing the order spreads this bias evenly across the two models but does not remove judge-specific preferences.
- **Single seed.** Each configuration was trained and evaluated once, so small differences may reflect run-to-run variation. As a rough guide, with about 220 decisive pairs, a 95% interval around a 50% win rate is about ± 6.6 points. This is an approximate uncertainty estimate, not a formal significance test.
- **Small evaluation sets.** The main comparisons use 300 prompts each, and the persona evaluation uses only 24, so persona results depend heavily on individual examples.
- **Heuristic refusal counts.** Refusal rates come from keyword matching and are a rough diagnostic, not a measure of safety.
- **Limited exploration.** I tested only two HH-RLHF learning rates and three LoRA settings, and kept β fixed at 0.1, so its effect was not measured.
- **Failed run.** The first LoRA-DPO run failed because the trainer split the model across both GPUs. I reran it unchanged on a single GPU.

Reproducibility: The Kaggle notebooks `notebooks/00`–`05`, `02b` and `04b/04c` reproduce training and generation on T4×2, and `scripts/run_all_judging.py` reruns the judging. Model checkpoints are stored in the Kaggle notebook outputs `pa2-01-sft`, `pa2-02-dpo-hh`, `pa2-02b-dpo-hh-lr5e-6`, `pa2-03-dpo-uf` and `pa2-04/04b/04c`. Table 9 lists where every other result is stored in the repository. All figures are generated from the logged metrics with `scripts/plot_metrics.py`.

Table 9: Location of the results behind every table and figure in this report.

Artifact	Repository path
Run configurations and training metrics	`results/runs/<run>/` (`config.json`, `metrics.jsonl`, `summary.json`)
Data statistics	`results/data_stats.json`
Generated responses	`results/generations/` (`alpaca`, `hh`, `persona`)
Blinded comparison pairs	`results/eval/*_pairs.jsonl`
Judgments, summaries and failure logs	`results/eval/*_judged.jsonl`, `*_summary.json`, `*_failures.jsonl`
All comparisons in one table	`report/results_table.md`
Per-comparison evidence (Section 6)	`report/analysis/`

References

[1] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[2] E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.

[3] L. Ben Allal et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737, 2025.

[4] Hugging Face. Transformers, TRL and PEFT documentation. `https://huggingface.co/docs`.

[5] N. Ding, Y. Chen, B. Xu, et al. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations. EMNLP, 2023.

[6] Y. Bai, A. Jones, K. Ndousse, et al. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862, 2022.

[7] G. Cui, L. Yuan, N. Ding, et al. UltraFeedback: Boosting Language Models with Scaled AI Feedback. ICML, 2024.

[8] Y. Dubois, B. Galambosi, P. Liang, T. B. Hashimoto. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv:2404.04475, 2024.

[9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, C. Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS, 2023.